Query Optimization

ISYS2099 | ISYS3434 · Database Applications · Week 3

Tom Huynh
School of Science, Engineering & Technology
RMIT University Vietnam



RMIT UNIVERSITY

Today's plan

01

Why optimize?

Symptoms of slow queries and the SQL pipeline

≈ 6 min

02

Cost-based optimization

Query trees and a worked I/O cost example

≈ 19 min

03

How query parts run

Access paths, sorting and three join algorithms

≈ 17 min

04

MySQL tools

Reading EXPLAIN and EXPLAIN ANALYZE

≈ 15 min

05

Optimization guides

Sargable queries, AND vs OR, joins vs subqueries

≈ 22 min

Then the tutorial: compare plans on two COMPANY databases (with and without keys): AND vs OR, join order, subqueries vs joins, sargable queries.



01

Why Optimize?

Part 01

Why it matters | Warning signs | SQL pipeline

RMIT UNIVERSITY

Why query optimization matters



Stay responsive as data grows

A query that is fine on 1,000 rows can crawl on 10 million.



Bad SQL hurts everyone

Slow pages, high server load and frustrated users.



Less I/O, CPU and memory

Optimized queries read fewer blocks and do less work.



Scale without new hardware

Efficient SQL beats constant hardware upgrades.

Last week's teaser: WHERE YEAR(Bdate) = 1990 ignores the index on Bdate. By the end of today you'll know why, and how to fix it.

Warning signs of poor query performance

Slower as tables grow	Response time rises sharply with table size.
Slow pages and timeouts	Users complain about database-driven pages.
Resources maxed out	High CPU, memory or disk use at peak times.
Simple SQL, long waits	A short query takes seconds or minutes.
Red flags in the logs	Repeated full table scans or temporary tables.



Diagnose before you treat

Find the slow query first, then ask MySQL how it runs it with EXPLAIN (Part 4).

Guessing and adding random indexes is not optimization.

How MySQL runs a query: parse, optimize, execute



SQL text

what you send



Parse

Syntax: valid SQL grammar?
Semantics: do the tables and columns exist?



Optimize

Generate candidate plans, estimate each one's cost from statistics (I/O and CPU), pick the cheapest



Execute

Run the plan through the storage engine and return the rows

```
SELEC name FROM Student;
```

→ **Error 1064: syntax error, stops at parse**

```
SELECT nme FROM Student;
```

→ **Error 1054: Unknown column 'nme' (name check)**

Today's focus: the optimizer, and how to help it choose a cheap plan.

Quiz: how does MySQL pick a plan?

QUIZ · 1 MIN · TYPE 1, 2 OR 3 IN THE CHAT

Two plans return the same rows. How does the optimizer decide which one to run?

1

It runs every candidate plan and keeps the fastest

2

It estimates each plan's cost from statistics and picks the cheapest

3

It always follows the order you wrote the SQL in

How to vote: think for 10 seconds, type 1, 2 or 3 in the chat, then press Enter when I count to three.



02

Cost-based Optimization

Part 02

Query trees

Relational algebra

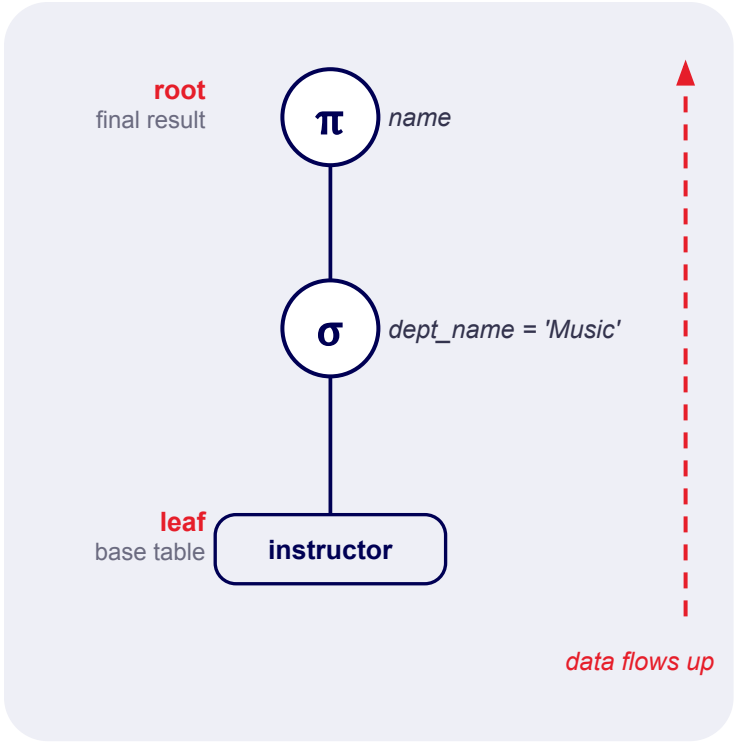
Worked I/O example

RMIT UNIVERSITY

Query trees: the shape of a query

- A **query tree** shows how a SQL query is broken into operations.
- Each internal node is an operation: **project, select (filter), join**, …
- **Leaf nodes** are base tables (or index access paths).
- Data flows **bottom-up**: the root produces the final result.
- It shows the **logical intent**, not necessarily how MySQL will execute it.

```
SELECT name FROM instructor
WHERE dept_name = 'Music';
```



Relational algebra: four symbols you need

Symbol	Operation	In SQL	Example
π	**PROJECT**	SELECT column list	π dirname (Director)
σ	**SELECT (filter rows)**	WHERE condition	σ dirname = 'Allen, Woody' (Director)
⋈	**JOIN**	JOIN … ON condition	Movie ⋈ m.dirnumb = d.dirnumb Director
×	**CARTESIAN PRODUCT**	FROM a, b with no condition	Movie × Director

More: en.wikipedia.org/wiki/Relational_algebra



Watch the names: σ (relational "select") is SQL's `WHERE`; SQL's `SELECT` list is π (project).

Quiz: from SQL to relational algebra

QUIZ · 1 MIN · TYPE 1, 2 OR 3 IN THE CHAT

Which one means SELECT dirname FROM Director WHERE dirnumb = 3?

1

σ dirname (π dirnumb = 3 (Director))

2

π dirnumb = 3 (σ dirname (Director))

3

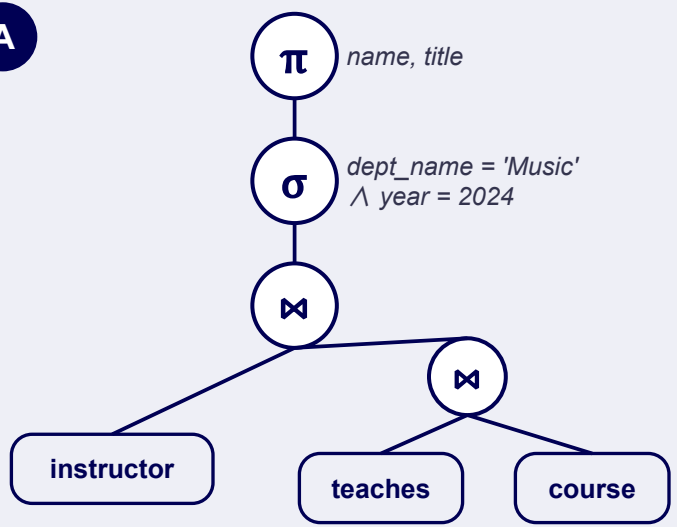
π dirname (σ dirnumb = 3 (Director))

How to vote: think for 10 seconds, type 1, 2 or 3 in the chat, then press Enter when I count to three.

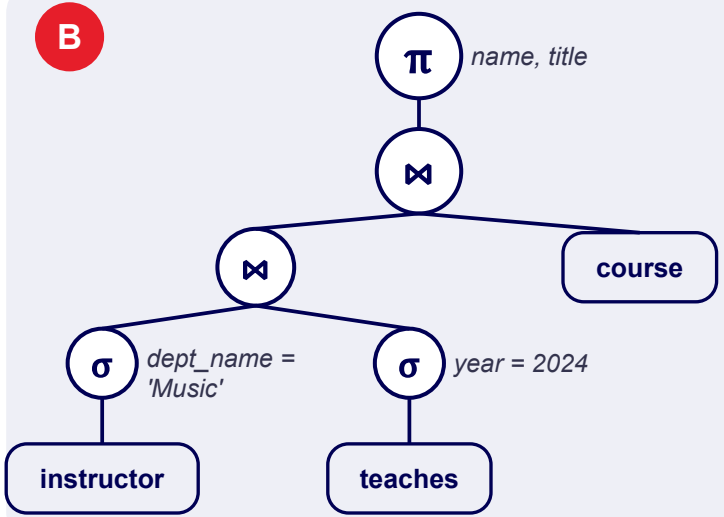
Solo activity: which query tree is better?

Names of instructors in the Music department who taught a course in 2024, with the course titles.

A



B



TASK (SOLO)
Compare trees A and B. Which one keeps the intermediate results smaller, and why?

TIME
2 min

PRODUCE
Post in the chat: A or B, plus a one-sentence reason

Worked example: Movie and Director

```
SELECT mvtitle
FROM Movie m JOIN Director d
  ON m.dirnumb = d.dirnumb
WHERE d.dirname = 'Allen, Woody';
```

Simplifying assumptions

- Each disk I/O reads or writes one record
- Every intermediate result is written back to disk
- No indexes on either table

Real systems move whole blocks and keep results in memory when they fit. The simplification makes the cost easy to count, and the comparison still holds.

Director 8 rows

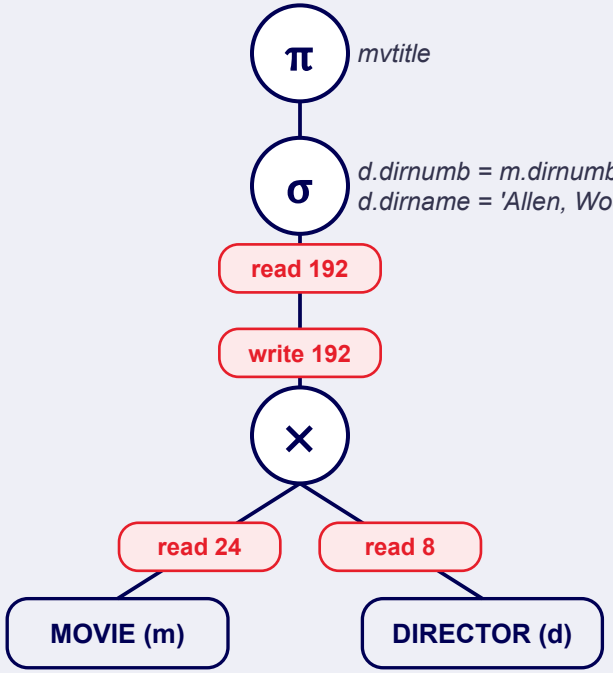


1 row is 'Allen, Woody'

Movie 24 rows

each movie has one director

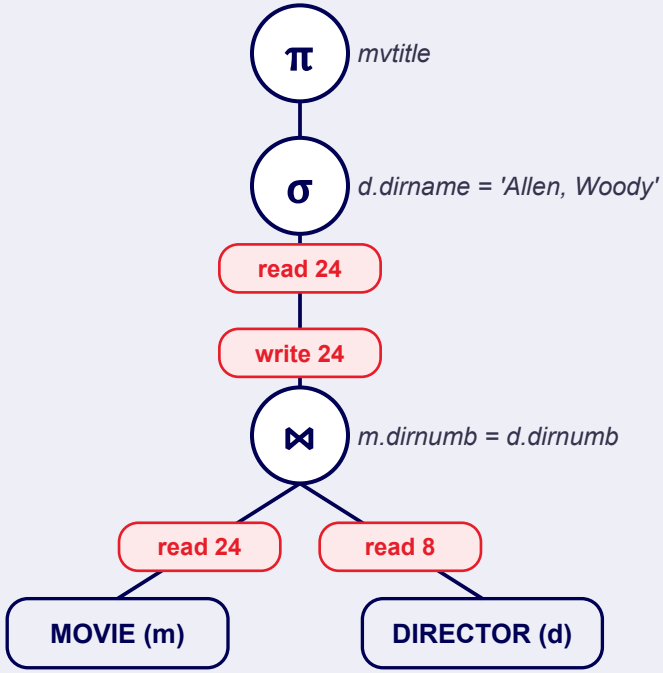
Tree 1: Cartesian product, then select, then project



Step	I/O
Read both tables: 24 + 8	32
Write the product: 24 × 8	192
Read it back for the select	192
Total I/O cost	**416**

Every movie is paired with every director, so the intermediate result has 192 rows, and only the rows for Woody Allen's movies survive the select.

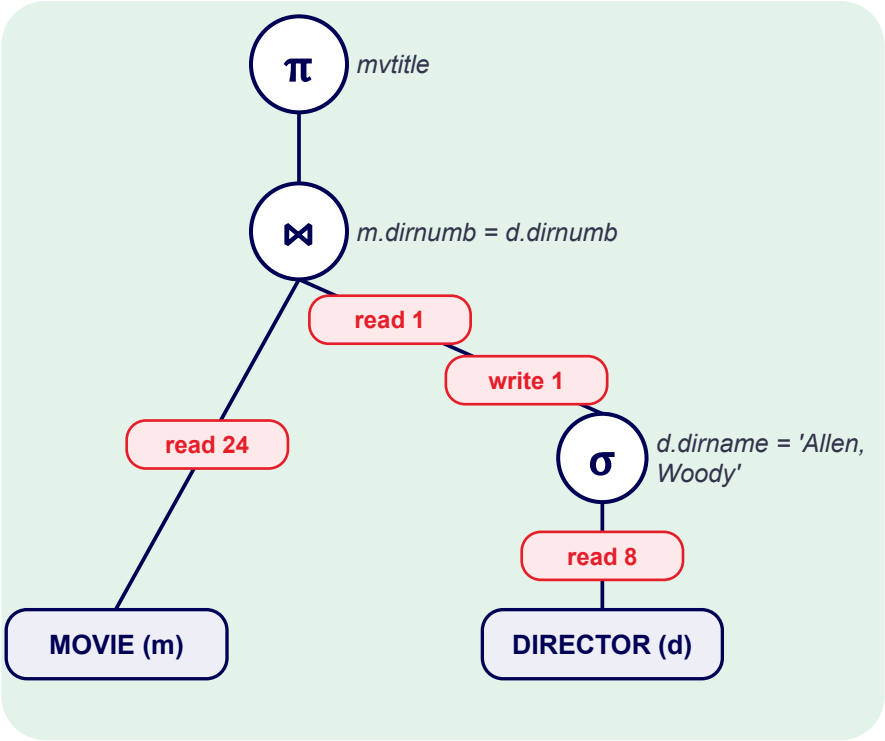
Tree 2: join, then select, then project



Step	I/O
Read both tables: 24 + 8	32
Write the join: 24 × 1	24
Read it back for the select	24
Total I/O cost	**80**

Think: what happens to the join cost if Movie and Director are too large to hold in main memory?

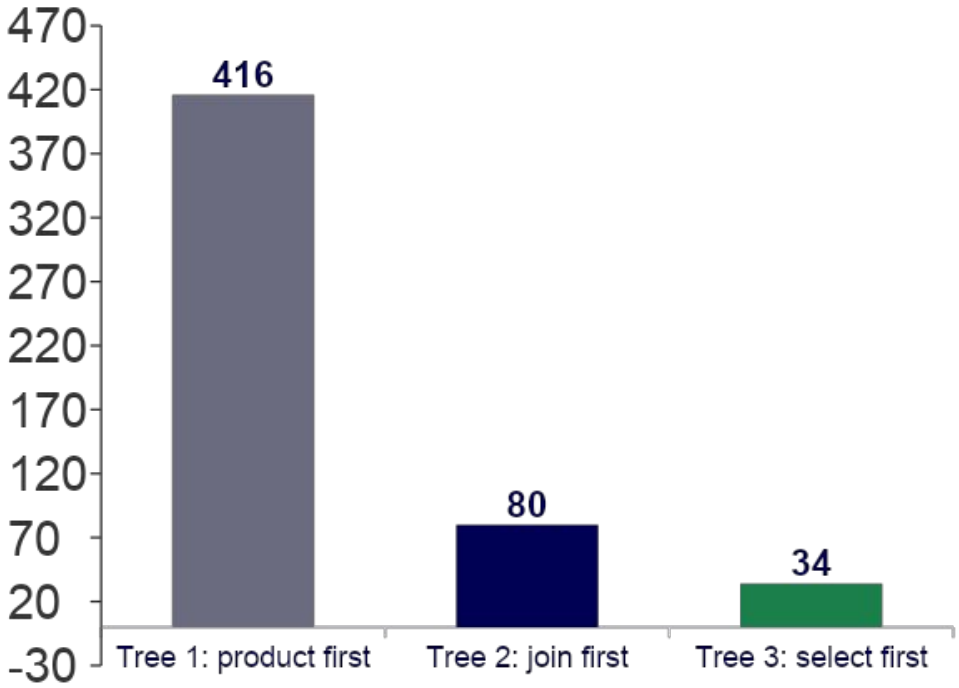
Tree 3: select first, then join, then project



Step	I/O
Read Director for the select	8
Write the 1 matching row	1
Join: read 1 + 24 rows	25
Total I/O cost	**34**

Filtering first shrinks Director from 8 rows to 1 before the join, so the join only pairs movies with one director.

Same answer, very different cost



12× less I/O

Tree 3 vs Tree 1, same result

Push selections down
filter rows as early as possible

Avoid Cartesian products
always join with a condition

Project early
carry only the columns you need

The optimizer explores these plans for you, using statistics to estimate the cost.

Solo activity: scale up the worked example

New table sizes

Movie

1,000 rows

Director

100 rows

1 row is 'Allen, Woody'

Same assumptions: 1 I/O per record, intermediate results written to disk, no indexes.

Tree 1: product first	I/O
Read both tables	?
Write the product	?
Read it back for the select	?
Total I/O cost	**?**

Tree 3: select first	I/O
Read Director for the select	?
Write the matching row	?
Join: read the match + all of Movie	?
Total I/O cost	**?**

Then: how many times cheaper is Tree 3? At the small sizes it was 12×.

TASK (SOLO)
Recount the I/O for Tree 1 and Tree 3 with the new table sizes. Fill in every step.

TIME
3 min

PRODUCE
Post in the chat: two totals, and the ratio Tree 1 / Tree 3

Quiz: what made Tree 3 cheapest?

QUIZ · 1 MIN · TYPE 1, 2 OR 3 IN THE CHAT

Tree 3 cost 34 I/Os against 416 for Tree 1. Which change made the biggest difference?

1

Filtering Director down to 1 row before the join

2

Building the Cartesian product first

3

Projecting mvtitle before reading the tables

How to vote: think for 10 seconds, type 1, 2 or 3 in the chat, then press Enter when I count to three.



03

How Query Parts Run

Part 03

WHERE access paths | Projection | ORDER BY | Join algorithms

RMIT UNIVERSITY

WHERE: choosing an access path

Full table scan

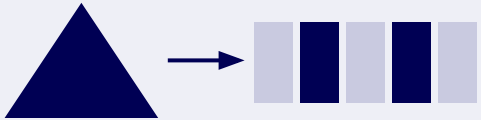
`EXPLAIN ALL`



Read every row and test the condition.

Index scan

`EXPLAIN ref · range`



Seek in the B+tree, then fetch the matching rows.

Index-only scan

`Extra: Using index`



Answer from the index alone, never touching the rows.

Equality test and an index on the attribute → **index lookup**

Comparison / range test and an ordered index (InnoDB uses B+trees) → **index range scan**

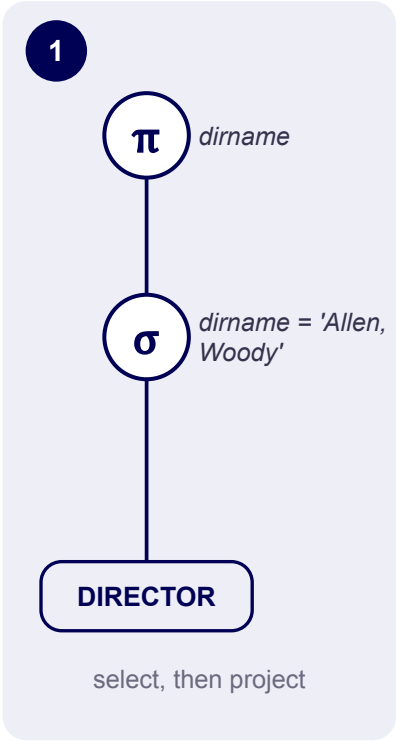
Complex test or no usable index → **full table scan**

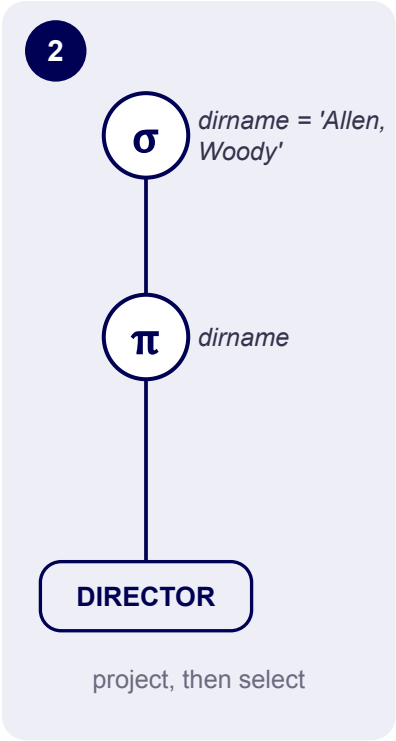
Projection (the SELECT list) is cheap

- Choosing columns is usually done **in memory** on rows already read: little or no extra disk I/O.
- But it still matters: **fewer columns** means smaller intermediate results and, with the right index, an index-only scan.

Which order is faster, in general?

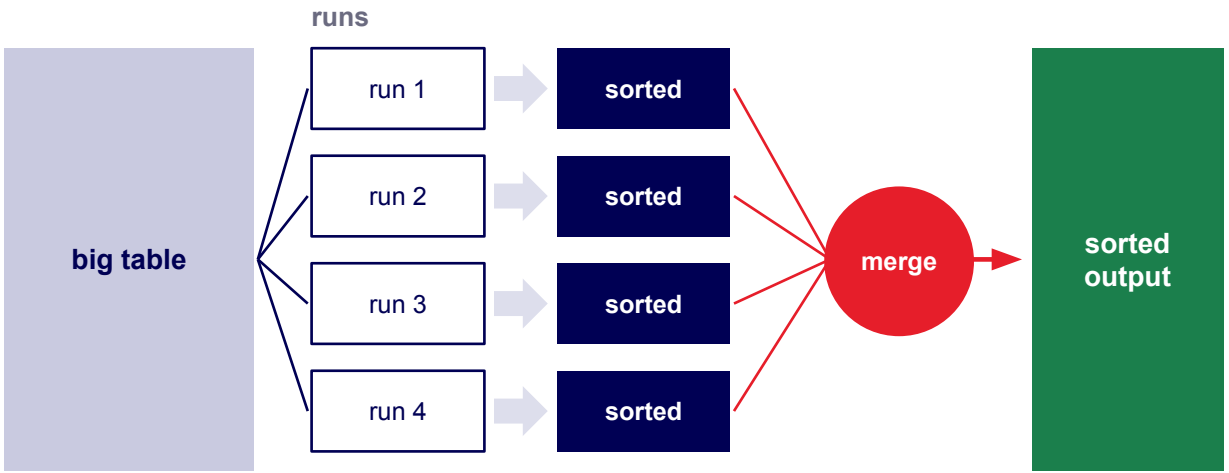
Both trees return the same names. Think about how many rows each operator has to handle.





ORDER BY: sorting is expensive

Fits in memory: a normal in-memory sort (e.g. quicksort). **Larger than memory:** an external merge sort.



1 **Split into runs that fit in memory**

2 **Sort each run, write it to a temp file**

3 **N-way merge of all runs**

Sort as late as possible
after WHERE has removed rows, so fewer rows are sorted

An index can remove the sort
B+tree leaves are already in key order (Week 2)

Spot it in EXPLAIN
Extra: "Using filesort" means MySQL is sorting

ORDER BY ... LIMIT 10: with and without an index

No index on stock

```
SELECT * FROM products
ORDER BY stock LIMIT 10;
```

EXPLAIN: type **ALL** · Extra **Using filesort**

```
-> Limit: 10 row(s)
   (actual time=39.1..39.1 rows=10 loops=1)
  -> Sort: products.stock,
     limit input to 10 row(s) per chunk
    -> Table scan on products
       (actual time=0.106..28.2 rows=100000 loops=1)
```

100,000
rows read, then sorted

39.1 ms
actual time

Index on price (idx_price)

```
SELECT * FROM products
ORDER BY price LIMIT 10;
```

EXPLAIN: type **index** · key **idx_price** · no filesort

```
-> Limit: 10 row(s)
   (actual time=0.0909..0.131 rows=10 loops=1)
  -> Index scan on products using idx_price
     (actual time=0.0901..0.13 rows=10 loops=1)
```

10
rows read, already in order

0.13 ms
actual time

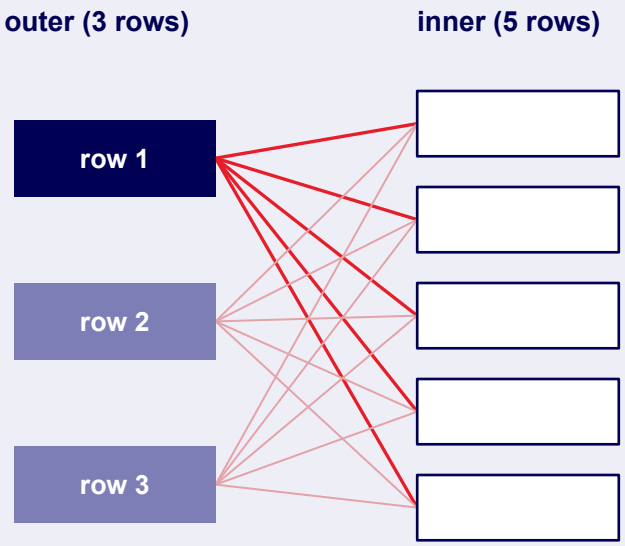
Real output, MySQL 8.0, on Week 2's 100,000-row products table. Cost estimates trimmed. Your times will differ; rows read will not.

Join algorithm 1: nested-loop join

```
for each row ro in outer_table do
  for each row ri in inner_table do
    if ro, ri satisfy the join condition
      add (ro, ri) to the result
  end
end
```

- One table is the **outer** (driving) table; the other is the **inner** table.
- Simple and works for **any** join condition, but one of the least efficient: **outer × inner** comparisons.

Index nested-loop: with an index on the inner table's join column, each inner pass becomes one index lookup. This is MySQL's usual join when indexes exist.

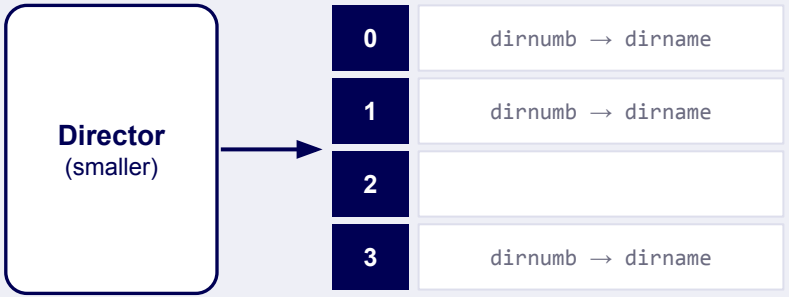


3 × 5 = 15 comparisons. With 10,000 × 10,000 rows: 100 million.

Join algorithm 2: hash join (equi-joins)

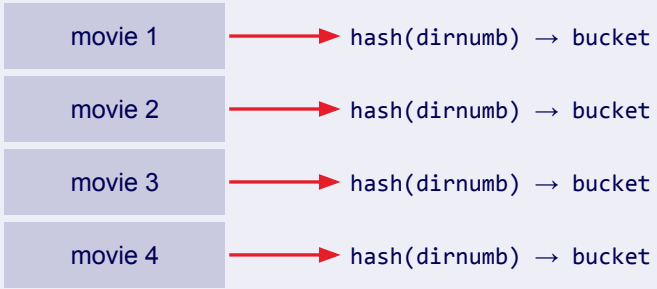
```
Movie m JOIN Director d ON m.dirnumb = d.dirnumb
```

1 **Build:** hash the smaller table on the join key



in-memory hash table

2 **Probe:** look up each row of the other table



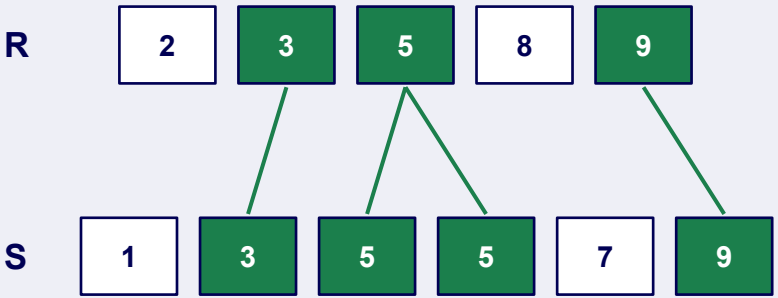
each row of Movie is read once

Too big for memory? Grace hash join: hash both tables into matching partitions with the same function, then join partition 1 with 1, 2 with 2, …

In MySQL 8.0.18+: used for equi-joins with no usable index. EXPLAIN shows `Using join buffer (hash join)`

Join algorithm 3: sort-merge join

Both inputs sorted on the join key: walk them together



Advance whichever pointer has the smaller key; matches pop out in one pass over each input.

- Much better than nested-loop when inputs are **already ordered** by the join attributes.
- Relations usually **aren't** sorted, so sorting first adds cost.
- Helps when one input is sorted and the other has an **index** on the join attribute.
- Also handy when the output must be sorted on the join key anyway.

Good to know: PostgreSQL, Oracle and SQL Server implement sort-merge joins. MySQL relies on nested-loop (with index lookups) and hash joins.

Three join algorithms at a glance

	Nested-loop	Hash join	Sort-merge
Join conditions	Any	Equality only	Equality (and some ranges)
Rough cost	outer × inner, or outer × index lookups	Read each table once (+ partition passes if too big)	Sort both (unless already sorted) + one merge pass
Best when	Inner table has an index on the join column, or tables are small	Large tables, no useful index, enough memory for the smaller one	Inputs already sorted on the join key
In MySQL	Yes: the default when indexes exist	Yes: MySQL 8.0.18 and later	Not used

You don't pick the algorithm: the optimizer does, based on indexes, sizes and statistics. You influence it with indexes and good SQL.

Solo activity: be the optimizer

1 Join 1

```
EMPLOYEE E JOIN DEPARTMENT D
  ON E.Dno = D.Dnumber
```

Dnumber is DEPARTMENT's PRIMARY KEY

EMPLOYEE has 8 rows

Your pick: _____

2 Join 2

```
clicks c JOIN sessions s
  ON c.session_id
   = s.session_id
```

2,000,000 clicks and 500,000 sessions

No index on session_id in either table

Your pick: _____

3 Join 3

```
orders o JOIN promotions p ON
  o.order_date BETWEEN
  p.start_date AND p.end_date
```

No indexes on the date columns

A range condition, not equality

Your pick: _____



TASK (SOLO)
For each join, pick what MySQL 8 will most likely use: index nested-loop, hash join or nested-loop.

TIME
2 min

PRODUCE
Post in the chat: three picks, each with a one-line reason

Quiz: why a hash join here?

QUIZ · 1 MIN · TYPE 1, 2 OR 3 IN THE CHAT

In company_no_key, EXPLAIN shows Extra: Using join buffer (hash join). Why?

1

MySQL is using the index on Dno

2

Both tables are already sorted on the join key

3

No index can serve the join, so MySQL builds a hash table in memory

How to vote: think for 10 seconds, type 1, 2 or 3 in the chat, then press Enter when I count to three.



04

MySQL Optimization Tools

Part 04

Why queries are slow

EXPLAIN

EXPLAIN ANALYZE

RMIT UNIVERSITY

Why is a query slow? Usual suspects

Scans too much data

Full table scans read far more rows than needed.

Missing or misused indexes

MySQL must examine many rows to find the few it needs.

Huge joins, no early filter

Joining big tables first creates huge intermediate results.

Functions on columns

WHERE YEAR(col) = … can stop MySQL using an index.

Sorting and grouping

ORDER BY, GROUP BY and temporary tables add work.

System-level limits

Locking, contention or I/O limits on the server.

EXPLAIN: ask MySQL for its plan

```
EXPLAIN
SELECT * FROM EMPLOYEE WHERE Dno = 4;
```

- Shows how MySQL **plans** to run a query, without running it.
- Access method (index or full scan), estimated rows, join type and order.
- Run it **before** you optimize: find full scans, missing indexes, bad joins.
- Works with **SELECT, DELETE, INSERT, REPLACE, UPDATE**.

A 3-step reading habit

1. `type`
   how is each table accessed? ALL = full scan
2. `key`
   which index was chosen? NULL = none
3. `rows × filtered`
   how many rows are read, and what share survives the WHERE

Reading EXPLAIN: columns and access types

Column	Meaning
id	SELECT identifier (one per SELECT in the query)
select_type	SIMPLE, PRIMARY, SUBQUERY, DEPENDENT SUBQUERY…
table	table used at this step; rows are in join order
partitions	partitions read (Week 4)
type	access type: see the ladder
possible_keys	indexes that could be used
key	index actually used
rows	estimated rows to examine
filtered	% of those rows left after the WHERE
Extra	e.g. Using where, Using index, Using filesort

type: best to worst

`const`	at most one row, matched on a PRIMARY KEY or UNIQUE index against constants
`eq_ref`	one row per row of the previous table, via a PRIMARY KEY or UNIQUE NOT NULL index
`ref`	several rows via a non-unique index (or prefix) with = or <=>
`range`	index range: <, >, BETWEEN, IN, LIKE 'abc%'
`index`	scans the whole index tree
`ALL`	full table scan

EXPLAIN: same query, two databases

```
EXPLAIN SELECT * FROM EMPLOYEE WHERE Dno = 4;
```

EXPLAIN column	company (with keys)	company_no_key
type	ref	ALL
possible_keys	Dno	NULL
key	Dno	NULL
rows	3	8
filtered	100.00	12.50
Extra	NULL	Using where



InnoDB built an index on the **foreign key Dno** automatically, so MySQL jumps straight to the 3 matching rows.



No index: MySQL reads **all 8 rows**. filtered 12.50 is a **guess** (1 in 8); really 3 of 8 match.

EXPLAIN on COMPANY: a key lookup and a join

1

```
EXPLAIN SELECT * FROM EMPLOYEE
WHERE Ssn = '123456789';
```

2

```
EXPLAIN SELECT * FROM DEPARTMENT
JOIN EMPLOYEE ON Dnumber = Dno
WHERE Dno = 4;
```

query	database	table	type	key	rows	Extra
1	company	EMPLOYEE	const	PRIMARY	1	NULL
1	company_no_key	EMPLOYEE	ALL	NULL	8	Using where
2	company	DEPARTMENT	const	PRIMARY	1	NULL
2	company	EMPLOYEE	ref	Dno	3	NULL
2	company_no_key	DEPARTMENT	ALL	NULL	3	Using where
2	company_no_key	EMPLOYEE	ALL	NULL	8	Using where; Using join buffer (hash join)

With keys: const and ref lookups, query 2 touches 4 rows. **Without keys:** two full scans and a hash join read 11 rows. Same answer either way.

Quiz: read one EXPLAIN row

QUIZ · 1 MIN · TYPE 1, 2 OR 3 IN THE CHAT

EXPLAIN shows: table EMPLOYEE · type ref · key Dno · rows 3. What happened?

1

MySQL used the Dno index to jump to the matching rows, estimating 3

2

MySQL read the whole table and kept 3 rows

3

MySQL found exactly one row through the primary key

How to vote: think for 10 seconds, type 1, 2 or 3 in the chat, then press Enter when I count to three.

EXPLAIN ANALYZE: plan plus reality

Since MySQL 8.0.18

Actually **runs** the statement, then shows the plan as a tree of iterators with:

estimates: `cost, rows`

reality: `actual time, rows, loops`

```
EXPLAIN ANALYZE SELECT ...;
```

It takes as long as the query itself, so be careful on huge or slow queries.

	EXPLAIN	EXPLAIN ANALYZE
Runs the query?	No	Yes
Shows	the estimated plan	plan + actual time and rows
Use it when	designing or debugging	validating after tuning, on production-like data

Estimated rows far from actual rows? The statistics may be out of date (refresh them with **`ANALYZE TABLE`**) or the plan may be poor.

Reading EXPLAIN ANALYZE output

```
EXPLAIN ANALYZE SELECT E.Lname, D.Dname    -- on company_no_key
FROM EMPLOYEE E JOIN DEPARTMENT D ON E.Dno = D.Dnumber;
```

```
-> Inner hash join (E.Dno = D.Dnumber)  (cost=3.2 rows=3)                    4
   (actual time=0.118..0.128 rows=8 loops=1)
    -> Table scan on E  (cost=0.117 rows=8)                                  3
       (actual time=0.00491..0.0119 rows=8 loops=1)
    -> Hash                                                                  2
        -> Table scan on D  (cost=0.55 rows=3)                               1
           (actual time=0.0609..0.0657 rows=3 loops=1)
```

1. **Read the innermost line first**
   scan D (3 rows)
2. **Build**
   hash D's rows on Dnumber
3. **Probe**
   scan E (8 rows), look up each Dno
4. **Join result**
   8 rows, 0.128 ms in total

`cost=... rows=...`
optimizer's estimate

`actual time=a..b`
ms to first row .. all rows

`rows=... loops=...`
actual rows; times this step ran

Spot the estimate miss: the join was estimated at `rows=3` but produced `rows=8`. With no index on Dno, MySQL has no statistics and guesses. Harmless here; on big tables a bad guess can mean a bad join order.

Solo activity: read the plan

```
EXPLAIN ANALYZE SELECT E.Lname, D.Dname FROM EMPLOYEE E JOIN DEPARTMENT D ON E.Dno = D.Dnumber;
```

Run on **company** *(with keys). Real output:*

```
-> Nested loop inner join  (cost=2.85 rows=8) (actual time=0.0508..0.0643 rows=8 loops=1)
    -> Covering index scan on D using Dname  (cost=0.55 rows=3) (actual time=0.023..0.0242 rows=3 loops=1)
    -> Index lookup on E using Dno (Dno=D.Dnumber)  (cost=0.589 rows=2.67)
       (actual time=0.0112..0.0123 rows=2.67 loops=3)
```

1. **Which join algorithm did MySQL use?**
2. **Which table is the outer (driving) table, and how is it read?**
3. **How many times did the lookup on E run, and how many rows did it return in total?**
4. **Compared with company_no_key on the previous slide, what replaced the hash join?**

TASK (SOLO)
Read the real plan above, from the innermost line out, and answer the four questions.

TIME
2 min

PRODUCE
Post in the chat: four short answers, one line each



05

Optimization Guides

Part 05

Sargable queries | LIKE & REGEXP | AND vs OR | Join order | Subqueries vs joins

RMIT UNIVERSITY

Sargable queries: let the index do the work

Sargable = "Search ARGument-able": the WHERE condition can be answered by searching an index.

✘ Not sargable	✔ Sargable
`WHERE YEAR(Bdate) = 1990`	`WHERE Bdate >= '1990-01-01'` `  AND Bdate <  '1991-01-01'`
`WHERE SQRT(myIntColumn) > 11.7`	`WHERE myIntColumn > 11.7 * 11.7`
`WHERE price * 1.1 > 110`	`WHERE price > 100`

Why? The index is sorted by Bdate, not by YEAR(Bdate). MySQL must compute the function for every row: a full scan.

Also watch implicit conversion: Ssn is CHAR(9), so `Ssn = 123456789` (a number) gives type ALL, key NULL. Quote it: `Ssn = '123456789'`.

Sargable or not: real output on 100,000 rows

Function on the column

```
SELECT * FROM products
WHERE DATE(created_at) = '2026-10-01';
```

EXPLAIN: type **ALL** · key **NULL**

```
-> Filter: (cast(products.created_at as date)
             = '2026-10-01')
   (actual time=0.237..37.4 rows=289 loops=1)
  -> Table scan on products
     (actual time=0.0875..30.7 rows=100000 loops=1)
```

100,000

rows read to return 289

37.4 ms

actual time

Range on the bare column

```
SELECT * FROM products
WHERE created_at >= '2026-10-01'
  AND created_at < '2026-10-02';
```

EXPLAIN: type **range** · key **idx_created**

```
-> Index range scan on products using idx_created
   over ('2026-10-01 00:00:00' <= created_at
         < '2026-10-02 00:00:00')  (...)
   (actual time=0.16..1.27 rows=289 loops=1)
```

289

rows read to return 289

1.27 ms

actual time

Real output, MySQL 8.0, Week 2's products table plus CREATE INDEX idx_created ON products (created_at). Cost estimates trimmed.

Solo activity: fix the WHERE

1. `WHERE Salary + 5000 > 40000`
   index on Salary

2. `WHERE LEFT(Lname, 2) = 'Wa'`
   index on Lname

3. `WHERE Ssn = 987654321`
   Ssn is CHAR(9), the PRIMARY KEY

4. `WHERE DATE(created_at) >= '2026-10-01'`
   index on created_at

→ `WHERE` *your rewrite* · *type:* ____

→ `WHERE` *your rewrite* · *type:* ____

→ `WHERE` *your rewrite* · *type:* ____

→ `WHERE` *your rewrite* · *type:* ____

TASK (SOLO)
Rewrite each condition so it compares the bare column and still returns the same rows.

TIME
3 min

PRODUCE
Post in the chat: four sargable WHERE clauses, each with its expected EXPLAIN type

LIKE and REGEXP: patterns vs indexes

Condition	Uses a B+tree index?	Why
name LIKE 'abc%'	✔ yes (range)	known prefix: seek to "abc", read until it ends
name LIKE '%abc'	✘ no	match could start anywhere
name LIKE '%abc%'	✘ no	leading wildcard: full scan
name REGEXP '^abc'	✘ never	REGEXP never uses indexes
MATCH(name) AGAINST ('abc')	✔ FULLTEXT index	word search on large text (Week 2)

Avoid leading wildcards
on indexed columns

Use REGEXP sparingly
or only on small tables

Check with EXPLAIN
confirm the scan type

Quiz: which pattern can use the index?

QUIZ · 1 MIN · TYPE 1, 2 OR 3 IN THE CHAT

There is a B+tree index on email. Which condition lets MySQL seek in it?

1

```
email LIKE '%@rmit.edu.vn'
```

2

```
email LIKE 'minh%'
```

3

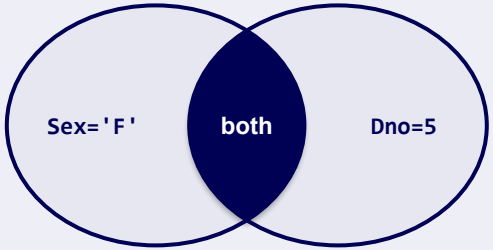
```
email REGEXP '^minh'
```

How to vote: think for 10 seconds, type 1, 2 or 3 in the chat, then press Enter when I count to three.

AND vs OR: how many indexes do you need?

AND `Sex = 'F' AND Dno = 5`

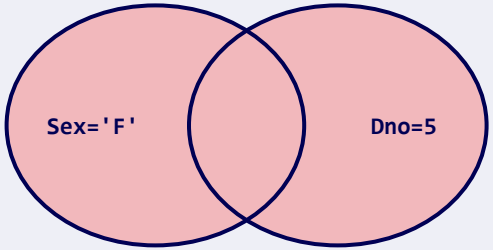
shaded = rows returned



One indexed condition is enough: MySQL seeks with it, then checks the other condition on those few rows.

OR `Sex = 'F' OR Dno = 5`

shaded = rows returned



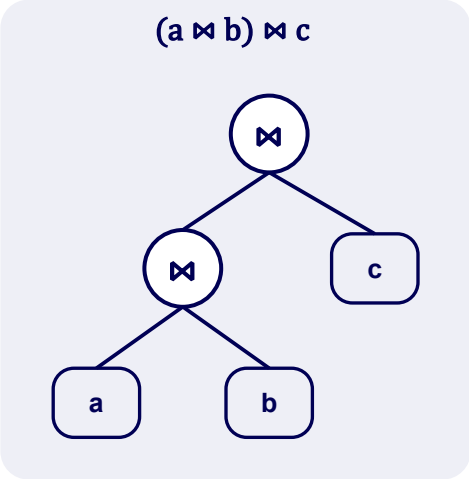
Rows can match either side. Every branch needs an index, or MySQL falls back to a full scan.

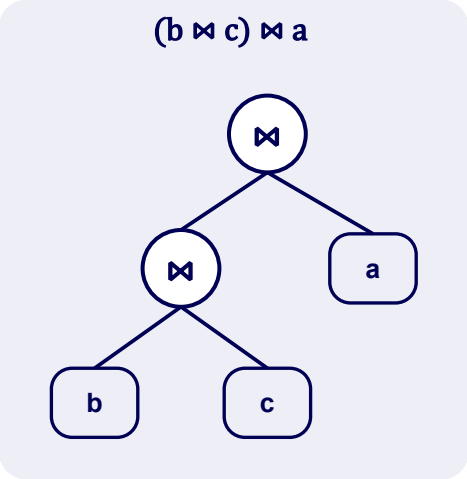
With indexes on both columns, **MySQL may use an index merge (EXPLAIN type: index_merge).**

Tutorial Ex 1: does swapping the order of the conditions change the plan? Predict first.

Join order: which tables first?

Tables a, b, c (a joins b, b joins c): two orders, same rows.





- The **optimizer chooses the order** of inner joins, whatever order you write them in.
- A good order starts from the **most selective** (most filtered) table and follows **indexed** join columns, keeping intermediate results small.
- EXPLAIN lists tables **in join order**.
- **`STRAIGHT_JOIN`** forces the written order: useful for experiments.

Tutorial Ex 2: write the employee, department and dependent query in both orders. Compare EXPLAIN on company and company_no_key: does the order MySQL uses change?

Rewriting correlated subqueries as JOINs

Correlated subquery conceptually runs once per outer row

```
SELECT c.id, c.name
FROM customers c
WHERE EXISTS (
  SELECT * FROM orders o
  WHERE o.customer_id = c.id);
```

JOIN rewrite optimizer can pick order and indexes

```
SELECT DISTINCT c.id, c.name
FROM customers c
JOIN orders o
  ON o.customer_id = c.id;
```

Rewriting may need DISTINCT (or GROUP BY)
a customer with 3 orders would appear 3 times

MySQL 8 often does this for you
it turns many IN and EXISTS subqueries into semi-joins

Check select_type in EXPLAIN
DEPENDENT SUBQUERY = runs per row: a warning sign

Tutorial Ex 3

Departments with employees who have dependents: write it with IN (…), then with JOIN, and compare the two plans.

EXISTS vs IN vs JOIN

	Typical use	Notes
EXISTS	Is there at least one match?	Can stop at the first match: good for presence checks
IN	Is the value in this list or subquery?	Can be slower with big subqueries; MySQL 8 often rewrites it as a semi-join
JOIN	Need columns from both tables	Most flexible; fast with indexes; may duplicate rows
NOT EXISTS / LEFT JOIN … IS NULL	Find rows with no match (anti-join)	Prefer these to NOT IN, which breaks if the subquery returns NULL

Customers without any orders (anti-join):

```
SELECT c.name FROM customers c
LEFT JOIN orders o ON c.id = o.customer_id
WHERE o.customer_id IS NULL;
```

Don't guess, measure: the optimizer may run EXISTS, IN and JOIN versions with the same plan. Compare them with EXPLAIN.

Common mistakes and better habits

✘ Mistake	✔ Better habit
SELECT * everywhere	Select only the columns you need
Join everything, filter at the end	Filter early so fewer rows are joined
Implicit conversions; comma joins	Match data types; write explicit JOIN … ON
Subqueries where a join is clearer	Use joins, then compare plans
"Our data is small"	Plan for growth: it won't stay small
Testing on 10 rows	Test with realistic data volumes

Tutorial this week: same data, two databases



`company`

Canvas: Company database with keys

- PRIMARY KEYs and FOREIGN KEYs
- InnoDB creates an index for every key
- Expect const, eq_ref, ref access



`company_no_key`

Canvas: Company database without keys

- Same tables and rows, no keys
- No indexes at all
- Expect ALL scans and hash joins

For every exercise: run EXPLAIN (or EXPLAIN ANALYZE) on both databases and write down type, key, rows and Extra for each table. The differences are the lesson.

Script links are on Canvas → W3 Tutorials.

Solo activity: predict before you run

Exercise	Task	Watch in EXPLAIN	Your prediction
1 AND vs OR	Female employees in dept 5 (AND); female employees and employees in dept 5 (OR); swap condition order	type, possible_keys, key, rows	
2 Join order	Employee, department and dependent names, written in two join orders	table order, type per table, hash join in Extra	
3 Subquery vs JOIN	Departments with employees who have dependents: nested IN, then JOIN	select_type, rows	
4 Sargable	Index Bdate; query an age range with YEAR(), then rewrite it	key: NULL vs the Bdate index; type ALL vs range	

TASK (SOLO)
Predict what EXPLAIN will show for each exercise, in company and in company_no_key.

TIME
3 min

PRODUCE
Post in the chat: one prediction per row. Keep a copy for the tutorial.

Quick check: type 1, 2 or 3 in the chat

Q1

Which WHERE clause can use an index on order_date?

1. `order_date >= '2024-01-01' AND order_date < '2025-01-01'`
2. `YEAR(order_date) = 2024`
3. `DATE_FORMAT(order_date, '%Y') = '2024'`

Q2

A query reads 100,000 rows to return 289. What do you check first?

1. **Add more memory to the server**
2. **Is the WHERE sargable and indexed?**
3. **Rewrite it as a subquery**

Q3

Two large tables are joined on equality and no index can serve the join. Which algorithm will MySQL 8 use?

1. **Index nested-loop**
2. **Sort-merge join**
3. **Hash join**

Key takeaways

The optimizer compares plans by cost

estimated from statistics; filter early, avoid Cartesian products

Joins run as nested-loop or hash joins

indexes on join columns make nested-loop fast

EXPLAIN shows the plan, ANALYZE the reality

read type, key, rows and Extra first

Write sargable, index-friendly SQL

no functions on indexed columns; measure rewrites

Next week · W4

Partitioning

Six years of sales, but most queries ask about one year. What if MySQL could skip the other five years entirely?

To do: register your project team

Thank you

Questions?

RMIT UNIVERSITY



Copyright © 2023 RMIT University Vietnam